

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0601

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO1: *Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)*

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks: 25

Annexure A

APPLICATION BASED QUESTIONS

Code of Conduct:

I S.KRISHNA CHAITHANYA(192424287) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.KRISHNA CHAITHANYA

11. In Shop X, there are 20 bottles of pure oil and 30 bottles of adulterated oil, while in Shop Y, there are 40 bottles of pure oil and 20 bottles of adulterated oil. A bottle is selected at random from one of the shops, with Shop Y being twice as likely to be chosen as Shop X. The selected bottle is found to be adulterated. (5 Marks)

CODE:

```
pure_X = 20
adulterated_X = 30
total_X = pure_X + adulterated_X
pure_Y = 40
adulterated_Y = 20
total_Y = pure_Y + adulterated_Y
P_X = 1 / 3
P_Y = 2 / 3
P_A_given_X = adulterated_X / total_X
P_A_given_Y = adulterated_Y / total_Y
P_A = (P_X * P_A_given_X) + (P_Y * P_A_given_Y)
P_X_given_A = (P_X * P_A_given_X) / P_A
P_Y_given_A = (P_Y * P_A_given_Y) / P_A
print("Probability of selecting Shop X:", P_X)
print("Probability of selecting Shop Y:", P_Y)
print("Probability of adulterated bottle from Shop X:", P_A_given_X)
print("Probability of adulterated bottle from Shop Y:", P_A_given_Y)
print("Total probability of selecting an adulterated bottle:", P_A)
print("\nProbability that the adulterated bottle came from Shop X =", P_X_given_A)
print("Probability that the adulterated bottle came from Shop Y =", P_Y_given_A)
```

OUTPUT:

```
Probability of selecting Shop X: 0.3333333333333333
Probability of selecting Shop Y: 0.6666666666666666
Probability of adulterated bottle from Shop X: 0.6
Probability of adulterated bottle from Shop Y: 0.3333333333333333
Total probability of selecting an adulterated bottle: 0.42222222222222217

Probability that the adulterated bottle came from Shop X = 0.4736842105263158
Probability that the adulterated bottle came from Shop Y = 0.5263157894736843
```

12. A warehouse has two sections. Section A contains 15 defective items and 35 non-defective items, while Section B contains 25 defective items and 25 non-defective items. One section is chosen at random, but Section A is three times as likely to be chosen as Section B. An item selected from the chosen section is found to be defective. (5 Marks)

CODE:

```
defective_A = 15
non_defective_A = 35
total_A = defective_A + non_defective_A
defective_B = 25
non_defective_B = 25
total_B = defective_B + non_defective_B
P_A = 3 / 4
P_B = 1 / 4
P_D_given_A = defective_A / total_A
P_D_given_B = defective_B / total_B
print("==== Warehouse Information =====")
print("Section A")
print("Defective Items    :", defective_A)
print("Non-Defective Items :", non_defective_A)
print("Total Items        :", total_A)
print("\nSection B")
print("Defective Items    :", defective_B)
print("Non-Defective Items :", non_defective_B)
print("Total Items        :", total_B)
print("\n==== Selection Probabilities =====")
print("Probability of choosing Section A =", P_A)
print("Probability of choosing Section B =", P_B)
print("\n==== Conditional Probabilities =====")
print("P(Defective | Section A) =", P_D_given_A)
print("P(Defective | Section B) =", P_D_given_B)
P_D = (P_A * P_D_given_A) + (P_B * P_D_given_B)
print("\nProbability of selecting a defective item =",
      round(P_D, 4))
P_A_given_D = (P_A * P_D_given_A) / P_D
print("\n==== Bayes' Theorem =====")
print("Formula:")
print("P(A|D) = [P(A) × P(D|A)] / P(D)")
print("\nCalculation:")
print("P(A|D) = (", P_A, " × ", P_D_given_A, ") / ", round(P_D, 4))
print("\nProbability that the defective item came from Section A =",
      round(P_A_given_D, 4))
```

```

print("\nPercentage =", round(P_A_given_D * 100, 2), "%")
print("\n===== Final Answer =====")
print("The probability that the defective item came from Section A is")
print(round(P_A_given_D, 4), "or", round(P_A_given_D * 100, 2), "%")

```

OUTPUT:

```

===== Warehouse Information =====
Section A
Defective Items      : 15
Non-Defective Items  : 35
Total Items          : 50

Section B
Defective Items      : 25
Non-Defective Items  : 25
Total Items          : 50

===== Selection Probabilities =====
Probability of choosing Section A = 0.75
Probability of choosing Section B = 0.25

===== Conditional Probabilities =====
P(Defective | Section A) = 0.3
P(Defective | Section B) = 0.5

Probability of selecting a defective item = 0.35

===== Bayes' Theorem =====
Formula:
P(A|D) = [P(A) × P(D|A)] / P(D)

Calculation:
P(A|D) = ( 0.75 * 0.3 ) / 0.35

Probability that the defective item came from Section A = 0.6429

Percentage = 64.29 %

===== Final Answer =====
The probability that the defective item came from Section A is
0.6429 or 64.29 %

```

13. In a binary classification problem using a deep learning model with a K-Nearest Neighbor classifier, the model produces True Positive (TP) = 120 and True Negative (TN) = 150. The dataset contains 1000 samples per class, and 20% of the data is used for testing. (5 Marks)

Find the following: Accuracy, Precision, Sensitivity (Recall), Specificity.

CODE:

```

TP = 120
TN = 150

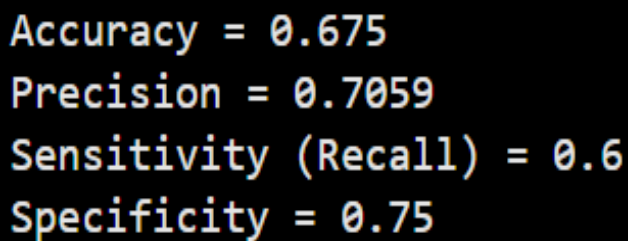
```

```

samples_per_class = 1000
total_samples = samples_per_class * 2
test_samples = int(total_samples * 0.20)
positive = test_samples // 2
negative = test_samples // 2
FN = positive - TP
FP = negative - TN
accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)
print("Accuracy =", round(accuracy, 4))
print("Precision =", round(precision, 4))
print("Sensitivity (Recall) =", round(recall, 4))
print("Specificity =", round(specificity, 4))

```

OUTPUT:



```

Accuracy = 0.675
Precision = 0.7059
Sensitivity (Recall) = 0.6
Specificity = 0.75

```

14.A medical image classification system using a deep learning framework combined with K-Nearest Neighbor (KNN) yields TP = 180 and TN = 160. Each class contains 1200 samples, and 25% of the dataset is used as test data.

Calculate: Accuracy Precision Sensitivity Specificity.

CODE:

```

TP = 180
TN = 160
samples_per_class = 1200
total_samples = samples_per_class * 2
test_samples = int(total_samples * 0.25)
positive = test_samples // 2
negative = test_samples // 2
FN = positive - TP
FP = negative - TN

```

```
accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)
print("Accuracy =", round(accuracy, 4))
print("Precision =", round(precision, 4))
print("Sensitivity (Recall) =", round(recall, 4))
print("Specificity =", round(specificity, 4))
```

OUTPUT:

```
Accuracy = 0.5667
Precision = 0.5625
Sensitivity (Recall) = 0.6
Specificity = 0.5333
```

15. A deep learning model is trained for a classification task, and the following loss values are observed: (5 Marks)

- i. Epoch 1: Training Loss = 2.8, Validation Loss = 2.6**
- ii. Epoch 2: Training Loss = 2.4, Validation Loss = 2.2**
- iii. Epoch 3: Training Loss = 2.1, Validation Loss = 1.9**
- iv. Epoch 4: Training Loss = 1.9, Validation Loss = 1.8**
- v. Epoch 5: Training Loss = 1.7, Validation Loss = 1.9**
- vi. Epoch 6: Training Loss = 1.5, Validation Loss = 2.1**
- vii. Epoch 7: Training Loss = 1.3, Validation Loss = 2.4**
- viii. Epoch 8: Training Loss = 1.2, Validation Loss = 2.7**

CODE:

```
import matplotlib.pyplot as plt
epochs = [1, 2, 3, 4, 5, 6, 7, 8]
training_loss = [2.8, 2.4, 2.1, 1.9, 1.7, 1.5, 1.3, 1.2]
```

```
validation_loss = [2.6, 2.2, 1.9, 1.8, 1.9, 2.1, 2.4, 2.7]
plt.plot(epochs, training_loss, marker='o', label='Training Loss')
plt.plot(epochs, validation_loss, marker='s', label='Validation Loss')
plt.title("Training Loss vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()

best_epoch = validation_loss.index(min(validation_loss)) + 1
print("Best Epoch:", best_epoch)
print("Minimum Validation Loss:", min(validation_loss))
if validation_loss[-1] > validation_loss[best_epoch - 1]:
    print("The model starts overfitting after Epoch", best_epoch)
```

OUTPUT:



```
Best Epoch: 4  
Minimum Validation Loss: 1.8  
The model starts overfitting after Epoch 4
```